

SKILLS

Extend Claude with skills

 Copy page

Create, manage, and share skills to extend Claude's capabilities in Claude Code. Includes custom commands and bundled skills.

Skills extend what Claude can do. Create a `SKILL.md` file with instructions, and Claude adds it to its toolkit. Claude uses skills when relevant, or you can invoke one directly with `/skill-name`.

Create a skill when you keep pasting the same instructions, checklist, or multi-step procedure into chat, or when a section of `CLAUDE.md` has grown into a procedure rather than a fact. Unlike `CLAUDE.md` content, a skill's body loads only when it's used, so long reference material costs almost nothing until you need it.

❗ For built-in commands like `/help` and `/compact`, and bundled skills like `/debug` and `/code-review`, see the [commands reference](#).

Custom commands have been merged into skills. A file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way. Your existing `.claude/commands/` files keep working. Skills add optional features: a directory for supporting files, frontmatter to [control whether you or Claude invokes them](#), and the ability for Claude to load them automatically when relevant.

Claude Code skills follow the [Agent Skills](#) open standard, which works across multiple AI tools. Claude Code extends the standard with additional features like [invocation control](#), [subagent execution](#), and [dynamic context injection](#). See [Using skill frontmatter outside Claude Code](#) for which frontmatter fields are part of the standard and which are Claude Code extensions.

give Claude detailed instructions and let it orchestrate the work using its tools. Most built-in commands instead execute fixed logic directly.

You invoke a bundled skill the same way as any other skill, by typing `/` followed by the skill name. Claude invokes some bundled skills automatically when relevant; others, including `/verify`, run only when you invoke them, which keeps you in control of when these longer-running checks spend time and tokens.

Bundled skills are available in every session. To turn them off, use the `disableBundledSkills` setting, which disables every bundled skill except `/doctor`.

❗ The `/doctor` setup checkup stays typable when `disableBundledSkills` is on, in Claude Code v2.1.205 and later. To hide it, set the `DISABLE_DOCTOR_COMMAND` environment variable or a `skillOverrides` entry of `"doctor": "off"`. Before v2.1.205, `/doctor` was a built-in command rather than a bundled skill.

Bundled skills are listed alongside built-in commands in the [commands reference](#), marked **Skill** in the Purpose column.

Run and verify your app

Three bundled skills work together to launch your app and confirm changes against the running app instead of just tests:

Skill	Purpose
<code>/run</code>	Launch and drive your app to see a change working
<code>/verify</code>	Build and run your app to confirm a code change does what it should, without falling back to tests or type checks
<code>/run-skill-generator</code>	Teach <code>/run</code> and <code>/verify</code> how to build and launch your project

`/run-skill-generator` records the recipe instead. It gets your app running from a clean environment, captures what worked (the install commands, the env vars, the launch script), and commits it as a per-project skill at `.claude/skills/run-<name>/`. After that, `/run`, `/verify`, and any other agent in the repo follow the recorded recipe instead of rediscovering it. Run `/run-skill-generator` once per project, and again if the build or launch process changes.

`/verify` can also record its own recipe. When it has to build and drive your app without a recorded recipe, it writes what worked to `.claude/skills/verify/SKILL.md` at the repo root, or in the touched package directory in a monorepo, so later runs and other agents follow the same steps. At the repo root, the recorded skill replaces the bundled `/verify`. This requires Claude Code v2.1.200 or later.

Claude edits the recorded file only when it steered a run wrong, such as a command that failed or a missing step, so you can commit the file without per-session diffs. Before v2.1.205, the bundled skill told Claude to fold in anything a run learned, which caused frequent merge conflicts.

Getting started

Create your first skill

This example creates a skill that summarizes the uncommitted changes in your git repository and flags anything risky. It pulls the live diff into the prompt before Claude reads it, so the response is grounded in your actual working tree rather than what Claude can guess from open files. Claude loads the skill automatically when you ask about your changes, or you can invoke it directly with `/summarize-changes`.

1 Create the skill directory

2 Write SKILL.md

Every skill needs a `SKILL.md` file with two parts: YAML frontmatter between `--` markers that tells Claude when to use the skill, and markdown content with the instructions Claude follows when the skill runs. The directory name becomes the command you type, and the `description` helps Claude decide when to load the skill automatically.

Save this to `~/.claude/skills/summarize-changes/SKILL.md` :

```
---
description: Summarizes uncommitted changes and flags anything risky. U
---

## Current changes

!`git diff HEAD`

## Instructions

Summarize the changes above in two or three bullet points, then list an
```

The `!`git diff HEAD`` line uses dynamic context injection: Claude Code runs the command and replaces the line with its output before Claude sees the skill content, so the instructions arrive with the current diff already inlined.

3 Test the skill

Open a git project, make a small edit to any file, and start Claude Code by running `claude` . You can test the skill two ways.

Or invoke it directly with the skill name:

```
/summarize-changes
```

Either way, Claude should respond with a short summary of your edit and a list of risks.

Where skills live

Where you store a skill determines who can use it:

Location	Path	Applies to
Enterprise	See managed settings	All users in your organization
Personal	<code>~/.claude/skills/<skill-name>/SKILL.md</code>	All your projects
Project	<code>.claude/skills/<skill-name>/SKILL.md</code>	This project only
Plugin	<code><plugin>/skills/<skill-name>/SKILL.md</code>	Where plugin is enabled

When skills share the same name, Claude Code resolves the conflict by source:

- Across levels, enterprise overrides personal, and personal overrides project.
 - For example, with a `deploy` skill in both `~/.claude/skills/` and your project's `.claude/skills/`, `/deploy` runs the personal one.

your skill.

- Plugin skills use a `plugin-name:skill-name` namespace, so they can't conflict with other levels.
 - For example, `my-plugin/skills/deploy/SKILL.md` becomes `/my-plugin:deploy` and loads alongside a `deploy` skill in your project's `.claude/skills/`.
- If you have files in `.claude/commands/`, those work the same way, but if a skill and a command share the same name, the skill takes precedence.
 - For example, with both `.claude/commands/deploy.md` and `.claude/skills/deploy/SKILL.md`, `/deploy` runs the skill.
- A skill or command from any of these sources overrides a skill **syncd from your claude.ai account** with the same name.
 - For example, with a `deploy` skill enabled on claude.ai and another in your project's `.claude/skills/`, `/deploy` runs the project one.

Skills also load from nested `.claude/skills/` directories below your working directory. When Claude reads or edits a file in a subdirectory, skills from that subdirectory's `.claude/skills/` become available. This lets a monorepo package provide its own skills that apply when working on that package, even if the session started at the repo root.

If a nested skill shares a name with another skill, both stay available. For example, with a `deploy` skill at the project root and another in `apps/web/.claude/skills/`:

- The nested one appears under a directory-qualified name, `apps/web:deploy`.
- Its description says which directory it applies to.
- Claude picks the variant that matches the files it is working on.

Typing `/deploy` runs the project-root skill. Type the qualified name `/apps/web:deploy` to run the nested variant explicitly.

invoked.

The folder name `synced` is reserved in the enterprise, personal, and project skills locations, in any capitalization. Claude Code **downloads the skills you enable on `claude.ai`** into `~/.claude/skills/synced/` when `CLAUDE_CODE_SYNC_SKILLS` is set in non-interactive mode, and skips a skill you author at that name.

A `<skill-name>` entry in the enterprise, personal, or project locations can be a symlink to a directory elsewhere on disk. Claude Code follows the symlink and reads `SKILL.md` from the target directory, and if the same target is reachable from more than one location, Claude Code loads the skill once. Plugin skills handle symlinks differently; see **Share files within a marketplace with symlinks**.

❗ Add a `.claude-plugin/plugin.json` to a skill folder and it loads as a **plugin** named `<name>@skills-dir`, so it can bundle agents, hooks, and MCP servers. In a project's `.claude/skills/`, this requires accepting the workspace trust dialog first.

Live change detection

Claude Code watches skill directories for file changes. When you add, edit, or remove a skill under `~/.claude/skills/`, the project `.claude/skills/`, or a `.claude/skills/` inside an `--add-dir` directory, Claude Code picks up the change within the current session, without a restart. If you create a top-level skills directory that didn't exist when the session started, restart Claude Code so it can watch the new directory.

❗ Live change detection covers `SKILL.md` text only. For a skill folder that is also a **plugin**, changes to `hooks/`, `.mcp.json`, `agents/`, and `output-styles/` need `/reload-plugins` to take effect.

Discovery from parent and nested directories

Project skills load from `.claude/skills/` in the directory where you start Claude Code and in every parent directory up to the repository root. Starting Claude in a subdirectory

Skills in nested `.claude/skills/` directories below your starting directory aren't loaded at startup. They load the first time Claude reads or edits a file inside that subdirectory, and stay available for the rest of the session. For example, after Claude edits a file under `packages/frontend/`, skills in `packages/frontend/.claude/skills/` become available. Until then, those skills don't appear in autocomplete and can't be invoked by name.

❗ Files in `.claude/commands/` support the same **frontmatter**, except `name` and `paths`, which Claude Code ignores in a command file. You invoke a command file by its file name. Skills are recommended since they support additional features like [supporting files](#).

Skills from additional directories

The `--add-dir` flag and `/add-dir` command **grant file access** rather than configuration discovery, but skills and commands are an exception: Claude Code loads `.claude/skills/` and `.claude/commands/` from each added directory automatically. This exception applies to `--add-dir`, `/add-dir`, and directories the Agent SDK adds through `additionalDirectories` in TypeScript or `add_dirs` in Python, which the SDK passes to Claude Code as `--add-dir`.

The `permissions.additionalDirectories` setting in `settings.json` grants file access only and doesn't load skills, commands, or subagents, even though the TypeScript option has the same name. See [Live change detection](#) for how skill edits are picked up during a session.

Claude Code loads skills, commands, and subagents from an added directory only when the project **setting source** is enabled, which is the default. If you pass `--setting-sources` on the CLI, or set `settingSources` or `setting_sources` explicitly in the SDK, include `project` in the list. In `--safe-mode`, Claude Code loads none of the three. A `strictPluginOnlyCustomization` managed policy and **bare mode** treat the three differently:

- **Subagents** in `.claude/agents/` : the policy's `agents` entry turns them off, not its `skills` entry. Bare mode skips every `.claude/agents/` folder, including the project's own.

Claude Code never watches `.claude/agents/` or `.claude/commands/` in an added directory, so after you add or edit a subagent or command file there, restart the session to load the change. In bare mode, Claude Code doesn't watch skill directories at all.

Apart from the `enabledPlugins` and `extraKnownMarketplaces` keys in an added directory's `.claude/settings.json` and `.claude/settings.local.json`, Claude Code doesn't load other `.claude/` configuration, such as output styles, from additional directories. See the [exceptions table](#) for the complete list of what is and isn't loaded, and the recommended ways to share configuration across projects.

❗ `CLAUDE.md` files from `--add-dir` directories are not loaded by default. To load them, set `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1`. See [Load from additional directories](#).

Skills in Cowork and cloud sessions

Cowork sessions and **cloud sessions**, including **routines**, don't read

`~/ .claude/skills/` on your machine. Both interactive and scheduled Cowork sessions load the skills enabled for your claude.ai account, synced at session start; manage them from **Customize** in the Desktop app sidebar or from the skills settings on claude.ai. Cloud sessions additionally load project skills committed to the cloned repository's `.claude/skills/`.

If a skill exists only in `~/ .claude/skills/` on your machine, Claude Code reports that the skill was not found when a **routine** invokes it, because each routine run starts as a fresh remote session. To make a personal skill available in these sessions:

- For Cowork and cloud sessions, enable the skill for your claude.ai account.

Desktop scheduled tasks are different: they run locally on your machine and load skills from the same locations as any other local session.

Skills synced from claude.ai

This section applies to you if you enabled skills for your claude.ai account. In Cowork and cloud sessions, Claude Code loads those skills without any setup on your machine. In any other session on your machine, Claude Code loads them only after you turn syncing on with `CLAUDE_CODE_SYNC_SKILLS` in a non-interactive run, as **Where synced skills load** describes.

Claude Code downloads a synced skill from your account rather than reading a file you wrote on the machine where the session runs, so it applies rules to synced skills that don't apply to the skills you store in the **skills locations**.

Where synced skills load

In a Cowork or cloud session, Claude Code loads the skills enabled for your claude.ai account, and **Skills in Cowork and cloud sessions** says how to choose which skills those sessions get.

In any other session on your machine, Claude Code loads them only after you download them once in a non-interactive run:

1 Enable the skills for your claude.ai account

Enable each skill you want for your claude.ai account, as **Skills in Cowork and cloud sessions** describes. Claude Code downloads only the skills you enabled, and it needs your claude.ai sign-in to download them.

2 Run Claude Code in non-interactive mode with syncing turned on

```
CLAUDE_CODE_SYNC_SKILLS=1 claude -p "List the skills you have available"
```

Claude Code downloads the skills into `~/.claude/skills/synced/`, answers the prompt, and exits like any other non-interactive run. The downloaded skills stay on disk after it exits, so you don't need to keep the run open. Claude Code downloads skills only during a run with `CLAUDE_CODE_SYNC_SKILLS` set, so after you enable or change a skill on claude.ai, run the command again. To change how long the run waits for the sync before it answers the prompt, set

`CLAUDE_CODE_SYNC_SKILLS_WAIT_TIMEOUT_MS`.

3 Confirm the skills load in a local session

Start an interactive session, without `CLAUDE_CODE_SYNC_SKILLS` set, and run `/skills`. The menu lists the downloaded skills under `claude.ai sync`. Every local session you start afterwards with the same claude.ai sign-in loads them from `~/.claude/skills/synced/` too.

When a synced skill name matches another command

Claude Code skips a synced skill whose name matches any other command, and that other command runs. The other command can be a built-in command, a **bundled skill**, a skill at any **local level**, a plugin skill, a file in `.claude/commands/`, or an **MCP prompt**. Claude Code also reserves the names of its own built-in commands and bundled skills even when they're unavailable in your session, for example after you turn bundled skills off, so it skips a synced skill with one of those names too.

Claude Code labels synced skills so you can tell where they came from. The `/skills` menu and `/context` group synced skills under `claude.ai sync`, and the `/` command menu marks them as coming from claude.ai.

`claude.ai sync` label is how you tell the two apart.

How Claude Code handles the frontmatter of a synced skill

Claude Code applies two rules to a synced skill's frontmatter:

- Claude Code honors the frontmatter in every kind of session, so an `allowed-tools` grant goes through the normal permission flow.
- Claude Code sanitizes the display text the skill supplies, such as its description. It removes control characters, and in text that reaches Claude, such as the description, it also escapes angle brackets so the text can't imitate Claude Code's internal formatting.

How Claude Code handles the body of a synced skill

What Claude Code does with a synced skill's body depends on where the session runs:

- In a cloud session, the body keeps the behavior a local skill has, because the session runs in an isolated container.
- In a Cowork session on your desktop, the body keeps the behavior a local skill has, except that Claude Code replaces every `!` command line with the `disableSkillShellExecution` placeholder, as it does for every skill you supply there.
- In any other session on your machine, Claude Code doesn't run `! commands`, doesn't attach the files that `@` references name the way it does for a local skill, and doesn't substitute the `${CLAUDE_PROJECT_DIR}` and `${CLAUDE_SESSION_ID}` placeholders, so the `@` references and both placeholders reach Claude as literal text. A `!` command line reaches Claude as literal text too, or as that placeholder when `disableSkillShellExecution` is on.

Remove a skill

How you remove a skill depends on where it came from:

- **Enterprise skill:** an administrator deletes the skill's directory from `.claude/skills/` inside the managed settings directory, for example `/etc/claude-code/.claude/skills/<skill-name>/` on Linux.
- **Plugin skill:** disable or uninstall the plugin that provides it, from the `/plugin` menu or with `/plugin uninstall <plugin-name>@<marketplace-name>`. Claude Code unloads the plugin's skills after you run `/reload-plugins` or restart; see Apply plugin changes without restarting.
- **Skill synced from claude.ai:** turn the skill off for your claude.ai account, in the same place you enabled it. Claude Code removes it from `~/.claude/skills/synced/` the next time it syncs your skills. If you delete the directory by hand instead, the next sync downloads it again while the skill stays enabled on claude.ai.
- **Bundled skill:** set `disableBundledSkills` to `true` to turn off every bundled skill except `/doctor`, or set one skill to `"off"` in `skill0verrides` to hide it.

To keep a personal or project skill but stop Claude from invoking it on its own, set `disable-model-invocation: true` in its frontmatter, or `"user-invocable-only"` in `skill0verrides` when you don't want to edit the file.

Configure skills

Skills are configured through YAML frontmatter at the top of `SKILL.md` and the markdown content that follows.

Types of skill content

Skill files can contain any instructions, but thinking about how you want to invoke them helps guide what to include:

Reference content adds knowledge Claude applies to your current work. Conventions, patterns, style guides, domain knowledge. This content runs inline so Claude can use it alongside your conversation context.

When writing API endpoints:

- Use RESTful naming conventions
- Return consistent error formats
- Include request validation

Task content gives Claude step-by-step instructions for a specific action, like deployments, commits, or code generation. These are often actions you want to invoke directly with `/skill-name` rather than letting Claude decide when to run them. Add `disable-model-invocation: true` to prevent Claude from triggering it automatically. The example below adds `context: fork`, which runs the skill in its own subagent context; see [Run skills in a subagent](#).

```
---
name: deploy
description: Deploy the application to production
context: fork
disable-model-invocation: true
---
```

Deploy the application:

1. Run the test suite
2. Build the application
3. Push to the deployment target

Keep the body itself concise. Once a skill loads, its content stays in context across turns, so every line is a recurring token cost. State what to do rather than narrating how or why, and apply the same conciseness test you would for CLAUDE.md content.

```
---
name: my-skill
description: What this skill does
disable-model-invocation: true
allowed-tools: Read Grep
---
```

Your skill instructions here...

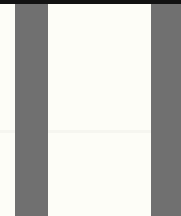
All fields are optional. Only `description` is recommended so Claude knows when to use the skill.

Boolean fields accept `yes`, `no`, `on`, `off`, `1`, and `0` in any letter case, in addition to `true` and `false`. Before v2.1.218, Claude Code recognized only `true` and `false`.

Field	Required	Description
<code>name</code>	No	Display name shown in skill listings. Defaults to the directory name. See How a skill gets its command name for how the field interacts with the name you type to invoke the skill.
<code>description</code>	Recommended	What the skill does and when to use it. Claude uses this to decide when to apply the skill. If omitted, uses the first paragraph of markdown content. Put the key use case first: the combined <code>description</code> and <code>when_to_use</code> text is truncated at 1,536 characters in the skill listing to reduce context usage.
<code>when_to_use</code>	No	Additional context for when Claude should invoke the skill, such as trigger phrases or example requests. Appended to <code>description</code> in the skill listing and counts toward the 1,536-character cap.
<code>argument-hint</code>	No	Hint shown during autocomplete to indicate expected arguments. Example: <code>[issue-number]</code> or <code>[filename] [format]</code> .

<code>disable-model-invocation</code>	No	Set to <code>true</code> to prevent Claude from automatically loading this skill. Use for workflows you want to trigger manually with <code>/name</code> . Also prevents the skill from being preloaded into subagents . As of v2.1.196, also prevents the skill from running when a scheduled task fires with the skill as its prompt. Default: <code>false</code> .
<code>user-invocable</code>	No	Set to <code>false</code> when only Claude should invoke the skill: Claude Code hides it from the <code>/</code> menu and doesn't run it when you type <code>/name</code> . Use for background knowledge users shouldn't invoke directly. Default: <code>true</code> .
<code>allowed-tools</code>	No	Tools Claude can use without asking permission during the turn that invokes this skill. The grant clears when you send your next message. Accepts a space- or comma-separated string, or a YAML list. See <u>Pre-approve tools for a skill</u> .
<code>disallowed-tools</code>	No	Tools removed from Claude's available pool while this skill is active. Use for autonomous skills that should never call certain tools, such as <code>AskUserQuestion</code> for a background loop. Accepts a space- or comma-separated string, or a YAML list. The restriction clears when you send your next message. Like deny rules, the field can't remove <code>EndConversation</code> while any other tool remains.
<code>model</code>	No	Model to use when this skill is active. The override applies for the rest of the current turn and is not saved to settings; the session model resumes on your next prompt. Accepts the same values as <code>/model</code> , or <code>inherit</code> to keep the active model. A value excluded by your organization's <code>availableModels</code> allowlist is not used and the session keeps its current model. With <code>context: fork</code> , the value sets the <u>forked subagent's model</u> instead, and an excluded value follows the <u>same rules as a subagent model override</u> .

context	No	Set to <code>fork</code> to run in a forked subagent context. See Run skills in a subagent .
agent	No	Which subagent type to use when <code>context: fork</code> is set.
background	No	Only applies with <code>context: fork</code> . Set to <code>false</code> to wait for the forked subagent's result in the turn that invoked the skill, instead of running it in the background . Default: <code>true</code> . Requires Claude Code v2.1.218 or later.
hooks	No	Hooks that Claude Code registers when the skill is invoked and keeps running for the rest of the session. See Hooks in skills and agents for the configuration format and the <code>once</code> option.
paths	No	Glob patterns that limit when this skill is activated. Accepts a comma-separated string or a YAML list. When set, Claude loads the skill automatically only when working with files matching the patterns. Uses the same format as path-specific rules .
shell	No	Shell to use for <code>!`command`</code> and <code>```!</code> blocks in this skill. Accepts <code>bash</code> (default) or <code>powershell</code> . Setting <code>powershell</code> runs inline shell commands via PowerShell when the PowerShell tool is enabled: it's on by default on Windows without Git Bash, on by default with Git Bash for claude.ai and Console accounts, and needs <code>CLAUDE_CODE_USE_POWERSHELL_TOOL=1</code> in Amazon Bedrock, Google Cloud's Agent Platform, and Microsoft Foundry sessions and on macOS, Linux, and WSL. Set it to <code>0</code> to turn the tool off.
metadata	No	Free-form YAML map for your own key-value data, such as entitlement or catalog fields, read by your own tooling from <code>SKILL.md</code> . Claude Code doesn't act on its contents, and drops a value that isn't a map. Don't reuse frontmatter field names such as <code>paths</code> as keys.



compatibility	No	Environment requirements for the skill, such as intended products or system prerequisites, as defined by the <u>Agent Skills</u> spec; see <u>Using skill frontmatter outside Claude Code</u> . Accepts a string of up to 500 characters. Claude Code accepts the field but doesn't act on it.
---------------	----	--

Using skill frontmatter outside Claude Code

Claude Code accepts every field in the table above. Outside Claude Code, you can use only the fields in the Agent Skills spec:

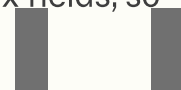
Distribution path	Frontmatter fields you can use
Claude Code skills at <u>any level</u> , including <u>plugin</u> skills	Every field in the table above
claude.ai skill uploads, the Skills API, and packaging with <code>package_skill.py</code> from <u>anthropics/skills</u>	<code>name</code> , <code>description</code> , <code>license</code> , <code>compatibility</code> , <code>metadata</code> , <code>allowed-tools</code>

When you enable a personal skill for Cowork and cloud sessions, including routines, you upload it to claude.ai, so the same rules apply.

If you include any field the spec doesn't allow, packaging or upload fails with a hard error instead of ignoring the field:

Unexpected key(s) in SKILL.md frontmatter: argument-hint. Allowed properties

Restricting frontmatter to the spec's six fields avoids the unexpected-key error above. The Agent Skills spec and the Skills API requirements define everything else those paths validate. Claude Code-only body features, such as dynamic context injection, don't function in claude.ai chat or through the API. Claude Code accepts all six fields, so frontmatter that follows the spec loads in Claude Code without changes.



directory name. In a plugin skill, `name` sets the last segment of the command and the plugin prefix stays in place.

The table below shows where the command name comes from for each layout:

Skill location	Command name source	Example
Skill directory under <code>~/.claude/skills/</code> or <code>.claude/skills/</code>	Directory name	<code>.claude/skills/deploy-staging/SKILL.md</code> → <code>/deploy-staging</code>
Nested <code>.claude/skills/</code> directory, when the name clashes with another skill	Subdirectory path relative to the working directory, then the skill directory name	<code>apps/web/.claude/skills/deploy/SKILL.md</code> → <code>/apps/web:deploy</code>
File under <code>.claude/commands/</code>	File name without extension	<code>.claude/commands/deploy.md</code> → <code>/deploy</code>
Plugin <code>skills/</code> subdirectory	Frontmatter <code>name</code> or the directory name, namespaced by plugin	<code>my-plugin/skills/review/SKILL.md</code> → <code>/my-plugin:review</code> , OR <code>/my-plugin:fancy</code> with <code>name: fancy</code>
Plugin root <code>SKILL.md</code>	Frontmatter <code>name</code> , with the plugin directory name as a fallback	<code>my-plugin/SKILL.md</code> with <code>name: review</code> → <code>/my-plugin:review</code> . See Path behavior rules

In a plugin skill, the frontmatter `name` replaces the directory name in the last segment of the command, so `my-plugin/skills/review/SKILL.md` with `name: fancy` becomes `/my-plugin:fancy`. The bare `/fancy` also invokes the skill unless another command already uses that name. If the `name` you write already starts with the plugin's own prefix, Claude Code doesn't add the prefix again on v2.1.246 or later. For example, `name: my-plugin:fancy` still becomes `/my-plugin:fancy`. From v2.1.216 through v2.1.245, Claude Code doubled the prefix when the `name` already carried it.

those sessions. In those sessions Claude Code also skips a synced skill named `help` or `feedback`, because it **skips a synced skill** whose name matches any built-in command whether or not that command can run. From v2.1.216 through v2.1.220, `help` and `feedback` were reserved too, so a plugin skill with one of those names was invocable only by its namespaced command in non-interactive sessions.

For a plugin-root `SKILL.md`, there is no skill directory to take the name from, so `name` supplies the whole final segment. Without a `name` field, Claude Code falls back to the plugin's directory name.

Available string substitutions

Skills support string substitution for dynamic values in the skill content:

Variable	Description
<code>\$ARGUMENTS</code>	All arguments passed when invoking the skill. When no placeholder receives an argument, Claude Code appends them as <code>ARGUMENTS: <value></code> . See <u>Pass arguments to skills</u> .
<code>\$ARGUMENTS[N]</code>	Access a specific argument by 0-based index, such as <code>\$ARGUMENTS[0]</code> for the first argument.
<code>\$N</code>	Shorthand for <code>\$ARGUMENTS[N]</code> , such as <code>\$0</code> for the first argument or <code>\$1</code> for the second.
<code>\$name</code>	Named argument declared in the <code>arguments</code> frontmatter list. Names map to positions in order, so with <code>arguments: [issue, branch]</code> the placeholder <code>\$issue</code> expands to the first argument and <code>\$branch</code> to the second.
<code>\${CLAUDE_SESSION_ID}</code>	The current session ID. Useful for logging, creating session-specific files, or correlating skill output with sessions.
<code>\${CLAUDE_EFFORT}</code>	The current effort level: <code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , or <code>max</code> . Ultracode is not a distinct level and reports as <code>xhigh</code> . Use this to adapt skill instructions to the active effort setting.

<code>`\${CLAUDE_PROJECT_DIR}`</code>	The project root directory. This is the same path hooks and MCP servers receive as <code>CLAUDE_PROJECT_DIR</code> . Use this to reference project-local scripts or files, such as <code>`\${CLAUDE_PROJECT_DIR}`/.claude/hooks/helper.sh</code> , independent of where the skill is installed.
<code>`\${CLAUDE_PLUGIN_ROOT}`</code>	The plugin's installation directory. Substituted only in plugin skills. Use this to reference scripts or files bundled anywhere in the plugin, including resources shared between the plugin's skills. See plugin environment variables .
<code>`\${CLAUDE_PLUGIN_DATA}`</code>	The plugin's persistent data directory , which survives plugin updates. Substituted only in plugin skills. Use this to reference installed dependencies, generated files, or caches that must outlive an update.

Claude Code substitutes ``${CLAUDE_SKILL_DIR}`` and ``${CLAUDE_PROJECT_DIR}`` in two places: the skill's markdown content, and Bash rules in the [allowed-tools](#) frontmatter. In a plugin skill, Claude Code substitutes ``${CLAUDE_PLUGIN_ROOT}`` and ``${CLAUDE_PLUGIN_DATA}`` in the same two places. Using the same variable in both places lets a skill run a bundled script without a permission prompt. The following skill shows the pattern:

```
---
name: render-chart
description: Render a chart from a CSV file
allowed-tools: Bash(`${CLAUDE_SKILL_DIR}/scripts/render.sh *)
---

Run `${CLAUDE_SKILL_DIR}/scripts/render.sh <csv-file>` to render the chart.
```

If this skill is installed at `~/.claude/skills/render-chart/`, both occurrences of ``${CLAUDE_SKILL_DIR}`` expand to that directory. The [allowed-tools](#) rule then matches the exact command the skill body tells Claude to run, so the script runs without prompting.

expands to `hello world` and `$1` to `second`. The `$ARGUMENTS` placeholder always expands to the full argument string as typed.

An indexed placeholder with no corresponding argument, such as `$2` when only one argument was passed, stays in the content unchanged. A named placeholder from the `arguments` frontmatter with no matching argument expands to an empty string.

If you pass an argument value that itself contains text such as `$1` or `$ARGUMENTS`, Claude Code inserts it as literal text and doesn't expand it. For example, if a skill's body contains `Summarize $0` and you run `/summarize "$ARGUMENTS from yesterday"`, Claude receives `Summarize $ARGUMENTS from yesterday`. Claude Code still replaces `${CLAUDE_*}` variables such as `${CLAUDE_SKILL_DIR}` after it inserts the arguments.

To include a literal `$` before a digit, `ARGUMENTS`, or a declared argument name, such as `$1.00` in prose, escape it with a backslash: `\$1.00`. A backslash before any other `$` is left unchanged. Only a single backslash directly before the token escapes it. A doubled backslash such as `\\$1` leaves both backslashes in place, and `$1` still expands to the argument value. The backslash escape covers only these argument placeholders. A backslash doesn't prevent substitution of a `${CLAUDE_*}` variable where the variable applies.

Example using substitutions:

```
---
name: session-logger
description: Log activity for this session
---

Log the following to logs/${CLAUDE_SESSION_ID}.log:

$ARGUMENTS
```

context every time the skill runs.

```
my-skill/
├─ SKILL.md (required - overview and navigation)
├─ reference.md (detailed API docs - loaded when needed)
├─ examples.md (usage examples - loaded when needed)
└─ scripts/
    └─ helper.py (utility script - executed, not loaded)
```

Reference supporting files from `SKILL.md` so Claude knows what each file contains and when to load it:

`## Additional resources`

- For complete API details, see `[reference.md](reference.md)`
- For usage examples, see `[examples.md](examples.md)`

💡 Keep `SKILL.md` under 500 lines. Move detailed reference material to separate files.

Control who invokes a skill

By default, both you and Claude can invoke any skill. You can type `/skill-name` to invoke it directly, and Claude can load it automatically when relevant to your conversation. Two frontmatter fields let you restrict this:

- `disable-model-invocation: true` : Only you can invoke the skill. Use this for workflows with side effects or that you want to control timing, like `/commit` , `/deploy` , or `/send-slack-message` . You don't want Claude deciding to deploy because your code looks ready.

This example creates a deploy skill that only you can trigger. If you set `disable-model-invocation: true`, Claude can't run the skill automatically:

```
---
name: deploy
description: Deploy the application to production
disable-model-invocation: true
---
```

Deploy \$ARGUMENTS to production:

1. Run the test suite
2. Build the application
3. Push to the deployment target
4. Verify the deployment succeeded

If Claude tries anyway, Claude Code blocks the call and instructs it not to reproduce the deploy steps another way, so expect Claude to suggest running `/deploy` yourself.

Here's how the two fields affect invocation and context loading:

Frontmatter	You can invoke	Claude can invoke	When loaded into context
(default)	Yes	Yes	Description always in context, full skill loads when invoked
<code>disable-model-invocation: true</code>	Yes	No	Description not in context, full skill loads when you invoke
<code>user-invocable: false</code>	No	Yes	Description always in context, full skill loads when invoked

Skill content lifecycle

When you or Claude invoke a skill, the rendered `SKILL.md` content enters the conversation as a single message and stays there across later turns. This persistence applies to the skill's instructions, not its permissions: an `allowed-tools` grant clears when you send your next message. Claude Code does not re-read the skill file on later turns, so write guidance that should apply throughout a task as standing instructions rather than one-time steps.

When Claude re-invokes a skill whose rendered content is identical to the copy already in context, Claude Code adds a short note that the skill is already loaded rather than a second copy of the content. When the rendered content differs, because the arguments changed or a **dynamic context** command produced new output, Claude Code appends the full content again.

Auto-compaction carries invoked skills forward within a token budget. When the conversation is summarized to free context, Claude Code re-attaches the most recent invocation of each skill after the summary, keeping the first 5,000 tokens of each. Re-attached skills share a combined budget of 25,000 tokens. Claude Code fills this budget starting from the most recently invoked skill, so older skills can be dropped entirely after compaction if you have invoked many in one session.

If a skill seems to stop influencing behavior after the first response, the content is usually still present and the model is choosing other tools or approaches. Strengthen the skill's `description` and instructions so the model keeps preferring it, or use **hooks** to enforce behavior deterministically. If the skill is large or you invoked several others after it, re-invoke it after compaction to restore the full content.

Pre-approve tools for a skill

The `allowed-tools` field grants permission for the listed tools during the turn that invokes the skill, so Claude can use them without prompting you for approval. The grant clears when you send your next message, even though the skill content **stays in context**

Workspace trust doesn't gate this field. Claude Code applies a project skill's `allowed-tools` whenever you or Claude invoke the skill, including in a `-p` run in a folder you've never trusted. A skill can grant itself broad tool access, so review the `allowed-tools` of skills checked into a repository before you run Claude Code there.

This skill lets Claude run git commands without per-use approval whenever you invoke it:

```
---
name: commit
description: Stage and commit the current changes
disable-model-invocation: true
allowed-tools: Bash(git add *) Bash(git commit *) Bash(git status *)
---
```

To remove tools from Claude's available pool while a skill is active, list them in `disallowed-tools` in the skill's frontmatter. The restriction clears when you send your next message. Like deny rules, the field can't remove `EndConversation` while any other tool remains. To block tools across all skills and prompts, add deny rules in your **permission settings**.

Pass arguments to skills

Both you and Claude can pass arguments when invoking a skill. Arguments are available via the `$ARGUMENTS` placeholder.

This skill fixes a GitHub issue by number. The `$ARGUMENTS` placeholder gets replaced with whatever follows the skill name:

```
Fix GitHub issue $ARGUMENTS following our coding standards.
```

1. Read the issue description
2. Understand the requirements
3. Implement the fix
4. Write tests
5. Create a commit

When you run `/fix-issue 123`, Claude receives “Fix GitHub issue 123 following our coding standards...”

If you invoke a skill with arguments but no placeholder in the skill’s content receives one, Claude Code appends `ARGUMENTS: <your input>` to the end of the skill content so Claude still sees what you typed. A placeholder is `$ARGUMENTS`, an indexed form such as `$1`, or a named argument. An indexed placeholder with no argument at its position stays as literal text and doesn’t count as receiving one. A named placeholder counts even when its position has no argument, because it expands to an empty string.

You can also stack several skills at the start of one message. Typing `/write-tests /fix-issue 123` loads both skills and passes the trailing text `123` as `$ARGUMENTS` to each of them. Before v2.1.199, only the first skill loaded and received `/fix-issue 123` as literal argument text.

Claude Code expands the first skill plus up to five more stacked after it. Expansion stops at the first token that isn’t an inline user-invocable skill, so a skill that runs as a **forked subagent**, such as `/code-review`, or one whose arguments may themselves start with a slash command, such as `/loop`, also ends the run there. That token and everything after it become the argument text for every expanded skill. `/code-review` runs as a forked subagent from v2.1.218; on earlier versions it ran inline and stacked.

To access individual arguments by position, use `$ARGUMENTS[N]` or the shorter `$N`:

```
Migrate the $ARGUMENTS[0] component from $ARGUMENTS[1] to $ARGUMENTS[2].
Preserve all existing behavior and tests.
```

Running `/migrate-component searchBar JavaScript TypeScript` replaces `$ARGUMENTS[0]` with `SearchBar`, `$ARGUMENTS[1]` with `JavaScript`, and `$ARGUMENTS[2]` with `TypeScript`. The same skill using the `$N` shorthand:

```
---
name: migrate-component
description: Migrate a component from one language to another
---
```

```
Migrate the $0 component from $1 to $2.
Preserve all existing behavior and tests.
```

Advanced patterns

Inject dynamic context

The `!`<command>`` syntax runs shell commands before the skill content is sent to Claude. The command output replaces the placeholder, so Claude receives actual data, not the command itself. Claude Code doesn't run these commands on your machine when the skill is **syncd from your claude.ai account.**

This skill summarizes a pull request by fetching live PR data with the GitHub CLI. The `!`gh pr diff`` and other commands run first, and their output gets inserted into the prompt:

```
agent: Explore
allowed-tools: Bash(gh *)
---

## Pull request context
- PR diff: !`gh pr diff`
- PR comments: !`gh pr view --comments`
- Changed files: !`gh pr diff --name-only`

## Your task
Summarize this pull request...
```

Substitution runs once over the original file. Command output is inserted as plain text and is not re-scanned for further `!`<command>`` placeholders, so a command cannot emit a placeholder for a later pass to expand.

The inline form is only recognized when `!` appears at the start of a line or immediately after whitespace. If `!` follows another character, as in `KEY=!`cmd``, the placeholder is left as literal text and the command does not run.

For multi-line commands, use a fenced code block opened with ````!` instead of the inline form:

```
## Environment
```!
node --version
npm --version
git status --short
```
```

To disable this behavior for skills and custom commands from user, project, plugin, or **additional-directory** sources, set `"disableSkillShellExecution": true` in **settings**. Each command is replaced with `[shell command execution disabled by policy]`

synced from your Claude.ai account, regardless of this setting. [How Claude Code](#)

handles the body of a synced skill says what Claude receives in place of the command in each kind of session.

💡 To request deeper reasoning when a skill runs, include `ultrathink` anywhere in the skill content. See [Use ultrathink for one-off deep reasoning](#).

How injected commands run

Claude Code picks the tool that runs a skill's injected commands from the `shell` key in the skill's frontmatter and your environment. Every combination runs the commands through the Bash tool or the PowerShell tool, except one that fails the invocation outright:

- `shell: powershell`, with the [PowerShell tool](#) enabled: the commands run through the PowerShell tool.
- `shell: bash` when bash isn't available: the invocation fails before any command runs. This happens on Windows without Git Bash. Claude Code shows `Skill <name> requires bash ('shell: bash' in frontmatter) but Git Bash was not found`.
- Any other combination: the commands run through the Bash tool when bash is available. When it isn't, they run through the PowerShell tool.

Either tool runs the commands the same way it runs Claude's own shell commands. They share the working directory, timeout, and output handling:

- **Working directory:** Claude Code runs each command in the session shell's current working directory. That directory moves when Claude runs `cd`. Use `${CLAUDE_SKILL_DIR}` or `${CLAUDE_PROJECT_DIR}` in paths that must resolve the same way every time.
- **stderr:** with the default `bash` shell, Claude Code merges stderr into stdout. Anything the command writes to stderr appears in the injected text.

auto-backgrounds, Claude Code kills it at the timeout. That failure aborts the invocation.

- **Output size:** output past the Bash tool's inline ceiling arrives as a file path plus a short preview, not truncated text. Output limits covers the ceiling and which variable adjusts which boundary.

The PowerShell tool applies the same timeout, backgrounding, and output-ceiling behavior to the commands it runs. See the PowerShell tool section for its specifics.

When an injected command fails

A failed command aborts the entire skill invocation, not just its own placeholder. Claude never sees the skill content for that invocation. The abort shows `Shell command failed for pattern "..."`. The error message includes the command's output under `[stderr]`.

With the default `bash` shell, any non-zero exit code counts as a failure. One carveout applies: Claude Code treats exit code 1 from search and comparison commands as a normal result and injects their output. Exit codes of 2 or higher fail even for those commands.

Which commands get the carveout depends on the shell:

- Default `bash` shell: the commands listed under Output limits
- `shell: powershell`, when the PowerShell tool is enabled: a different set that includes `grep` and `git diff` but not `find` or `diff`

With the default `bash` shell, append `|| true` to any other command you expect to exit non-zero. A check script that exits 1 when it finds problems is one example.

Injected commands never prompt for permission. When a command's permission check returns anything other than allow, Claude Code aborts the invocation. This includes a

`tools` . A matching ask or deny role can abort the invocation regardless of `allowed` tools . See [Manage permissions](#).

Run skills in a subagent

Add `context: fork` to your frontmatter when you want a skill to run in isolation. The skill content becomes the prompt that drives the subagent. It won't have access to your conversation history.

The forked subagent runs in the **background**: you keep working while it runs, and its result arrives in your conversation when it completes. Set `background: false` in the frontmatter to instead wait for the result in the turn that invoked the skill. Before v2.1.218, forked skills always blocked the turn until they finished.

Claude Code also waits for the result, even when the skill doesn't set `background: false` , in cases like these:

- In non-interactive mode, with the `-p` flag or the Agent SDK
- When you set `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` to `1` , which also turns off all other background task features
- When you invoke a forked skill while an earlier invocation of the same skill is still running
- When a **scheduled task** fires with the skill as its prompt

A backgrounded fork also runs with the **narrower tool set that applies to background subagents**: the skill's subagent is a regular agent type, so the exemption for subagents that fork the conversation doesn't cover it. If your skill's steps depend on a tool outside that set, set `background: false` to keep the full tool set.

A forked skill that runs in the background applies its edits outside your session's **checkpoints**, so `/rewind` doesn't undo them; use git to revert them.

Skills and subagents work together in two directions:

| Approach | System prompt | Task | Also loads |
|---|--------------------------|-----------------------------|---|
| Skill with <code>context: fork</code> | From agent type | SKILL.md content | CLAUDE.md, except when the agent is Explore or Plan |
| Subagent with <code>skills</code> field | Subagent's markdown body | Claude's delegation message | Preloaded skills + CLAUDE.md |

With `context: fork`, you write the task in your skill and pick an agent type to execute it. The built-in Explore and Plan agents skip CLAUDE.md and git status to keep their context small, so a forked skill using `agent: Explore` sees only the SKILL.md content and the agent's own system prompt. For the inverse, where you define a custom subagent that uses skills as reference material, see Subagents.

Example: Research skill using Explore agent

This skill runs research in a forked Explore agent. The skill content becomes the task, and the agent provides read-only tools optimized for codebase exploration:

```
---
name: deep-research
description: Research a topic thoroughly
context: fork
agent: Explore
---
```

Research \$ARGUMENTS thoroughly:

1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with specific file references

thoroughly...”)

3. The `agent` field determines the execution environment (model, tools, and permissions)
4. The subagent summarizes its results and returns them to your main conversation when it finishes

The `agent` field specifies which subagent configuration to use. Options include built-in agents (`Explore` , `Plan` , `general-purpose`) or any custom subagent from `.claude/agents/` . If omitted, uses `general-purpose` .

Restrict Claude’s skill access

By default, Claude can invoke any skill that doesn’t have `disable-model-invocation: true` set. Skills that define `allowed-tools` grant Claude access to those tools without per-use approval during the turn that invokes the skill; the grant clears when you send your next message. Your [permission settings](#) still govern baseline approval behavior for all other tools. A few built-in commands are also available through the Skill tool, including `/init` and `/security-review` . Other built-in commands such as `/compact` are not.

Three ways to control which skills Claude can invoke:

Disable all skills by denying the Skill tool in `/permissions` :

```
# Add to deny rules:
Skill
```

Allow or deny specific skills using [permission rules](#):

```
# Deny specific skills
Skill(deploy *)
```

Permission syntax: `Skill(name)` for exact match, `Skill(name *)` for prefix match with any arguments.

Hide individual skills by adding `disable-model-invocation: true` to their frontmatter. This removes the skill from Claude's context entirely.

❗ With `user-invocable: false`, you can't invoke the skill, but Claude still can. To keep Claude from invoking it through the Skill tool, set `disable-model-invocation: true`.

Override skill visibility from settings

The `skillOverrides` setting controls skill visibility from your settings instead of the skill's own frontmatter. Use it for skills whose SKILL.md you don't want to edit, such as ones checked into a shared project repo. The `/skills` menu writes it for you: highlight a skill and press `Space` to cycle states, then `Enter` to save to `.claude/settings.local.json`.

Each key is a skill name and each value is one of four states:

| Value | Listed to Claude | In <code>/</code> menu |
|------------------------------------|----------------------|------------------------|
| <code>"on"</code> | Name and description | Yes |
| <code>"name-only"</code> | Name only | Yes |
| <code>"user-invocable-only"</code> | Hidden | Yes |
| <code>"off"</code> | Hidden | Hidden |

The `/skills` menu labels the `"user-invocable-only"` state `user-only`.

A skill that is absent from `skill0overrides` is created as `on`. The example below collapses one skill to its name and turns another off entirely:

```
{
  "skill0overrides": {
    "legacy-context": "name-only",
    "deploy": "off"
  }
}
```

Plugin skills are not affected by `skill0overrides`. Manage those through `/plugin` instead.

Evaluate and iterate on a skill

Seeing a skill trigger tells you Claude found it, not that it did what you intended. To know a skill is working, measure two things separately: whether Claude invokes it on the prompts it should, and whether the output matches what you expect when it does.

The check for both is a baseline comparison. Collect a few realistic prompts, run each one in a fresh session with the skill available and again with it **disabled**, and compare the results. A fresh session matters because leftover context from authoring the skill will mask gaps in the written instructions.

Run evals with skill-creator

The `skill-creator` **plugin** automates the comparison loop inside Claude Code. Install it from the official marketplace:

```
/plugin install skill-creator@claude-plugins-official
```

install.

- The plugin is **not found in the marketplace**: check the plugin name.

If the install summary reports `Run /reload-plugins to activate.`, run that command to make the plugin's skills available in the current session. Then ask Claude to evaluate an existing skill, for example `evaluate my summarize-changes skill with skill-creator`. The plugin walks you through writing test cases and runs the loop:

- **Test cases**: stores prompts, input files, and expected behavior in `evals/evals.json` inside the skill directory
- **Isolated runs**: spawns a **subagent** per test case so each run starts with a clean context, and records token count and duration
- **Grading**: checks each assertion against the output and writes pass or fail with evidence to `grading.json`
- **Benchmark**: aggregates pass rate, time, and tokens for with-skill versus without-skill into `benchmark.json` so you can compare the pass-rate improvement against the token and time overhead
- **Version comparison**: runs a blind A/B between two versions of the skill so you can confirm an edit is an improvement before committing it
- **Description tuning**: generates should-trigger and should-not-trigger prompts, measures the hit rate, and proposes description edits when the skill activates on the wrong requests
- **Review viewer**: opens an HTML report where you inspect each output and record qualitative feedback that the next iteration reads

For the eval file format and the full iteration workflow, see [Evaluating skill output quality](#) on agentskills.io. For background on the benchmark and comparison modes, see the [skill-creator announcement](#).

- **Plugins:** Create a `skills/` directory in your plugin
- **Managed:** Deploy organization-wide through managed settings

Generate visual output

Skills can bundle and run scripts in any language, giving Claude capabilities beyond what's possible in a single prompt. One powerful pattern is generating visual output: interactive HTML files that open in your browser for exploring data, debugging, or creating reports.

This example creates a codebase explorer: an interactive tree view where you can expand and collapse directories, see file sizes at a glance, and identify file types by color.

Create the Skill directory:

```
mkdir -p ~/.claude/skills/codebase-visualizer/scripts
```

Save this to `~/.claude/skills/codebase-visualizer/SKILL.md`. The description tells Claude when to activate this Skill, and the instructions tell Claude to run the bundled script. The script path uses `${CLAUDE_SKILL_DIR}` so it resolves correctly whether the skill is installed at the personal, project, or plugin level:

```
---  
  
# Codebase Visualizer
```

Generate an interactive HTML tree view that shows your project's file structure

```
## Usage
```

Run the visualization script from your project root:

```
```bash  
python3 ${CLAUDE_SKILL_DIR}/scripts/visualize.py .
```
```

This creates 'codebase-map.html' in the current directory and opens it in your browser

```
## What the visualization shows
```

- ****Collapsible directories****: Click folders to expand/collapse
- ****File sizes****: Displayed next to each file
- ****Colors****: Different colors for different file types
- ****Directory totals****: Shows aggregate size of each folder

Save this to `~/ .claude/skills/codebase-visualizer/scripts/visualize.py` . This script scans a directory tree and generates a self-contained HTML file with:

- A **summary sidebar** showing file count, directory count, total size, and number of file types
- A **bar chart** breaking down the codebase by file type (top 8 by size)
- A **collapsible tree** where you can expand and collapse directories, with color-coded file type indicators

The script requires Python 3 but uses only built-in libraries, so there are no packages to install:

```
import sys
import webbrowser
from html import escape
```

... See all 133 lines

To test, open Claude Code in any project and ask “Visualize this codebase.” Claude runs the script, which prints the generated file’s path, such as `Generated /path/to/codebase-map.html`, and opens it in your browser. If you work in a headless environment where no browser opens, the printed path confirms the script succeeded.

This pattern works for any visual output: dependency graphs, test coverage reports, API documentation, or database schema visualizations. The bundled script does the work while Claude handles orchestration.

Troubleshooting

Skill not triggering

If Claude doesn’t use your skill when expected:

1. Check the description includes keywords users would naturally say
2. Verify the skill appears in `What skills are available?`
3. Try rephrasing your request to match the description more closely
4. Invoke it directly with `/skill-name` if the skill is user-invocable

If the frontmatter YAML is malformed, Claude Code loads the skill body with empty metadata, so `/skill-name` still works but Claude has no `description` to match against. Run with `--debug` to see the parse error.

To find `SKILL.md` files whose frontmatter doesn’t parse, run `claude plugin validate` on the skills directory, for example `claude plugin validate .claude/skills` for

If Claude uses your skill when you don't want it:

1. Make the description more specific
2. Add `disable-model-invocation: true` if you only want manual invocation

Skill descriptions are cut short

Claude Code loads a listing of skill names and descriptions into context so Claude knows what's available. The listing always contains every skill name, but if you have many skills, Claude Code shortens descriptions to fit the listing's character budget, which can strip the keywords Claude needs to match your request. The budget scales at 1% of the model's context window. When the listing overflows, Claude Code drops descriptions starting with the skills you invoke least, so the skills you use most keep their full text.

Run `/doctor` for an estimate of the listing's context cost and its biggest contributors. When the listing exceeds its budget, Claude Code also writes a warning to the debug log, visible with `--debug`.

The Skills row in `/context` reports the size of the listing after the budget is applied, so it matches what the model receives. Before v2.1.196, the row counted the full text of every description and could show a value several times larger than the configured budget.

To raise the budget, set the `skillListingBudgetFraction` setting (e.g. `0.02` = 2%) or the `SLASH_COMMAND_TOOL_CHAR_BUDGET` environment variable to a fixed character count. To free budget for other skills, set low-priority entries to `"name-only"` in `skillOverrides` so they list without a description. You can also trim the `description` and `when_to_use` text at the source: put the key use case first, since each entry's combined text is capped at 1,536 characters regardless of budget. The cap is configurable with `skillListingMaxDescChars`.

agentskills.io

- **Skill authoring best practices**: writing guidance that applies across Claude products
- **Subagents**: delegate tasks to specialized agents
- **Plugins**: package and distribute skills with other extensions
- **Hooks**: automate workflows around tool events
- **Memory**: manage CLAUDE.md files for persistent context
- **Commands**: reference for built-in commands and bundled skills
- **Permissions**: control tool and skill access
- **Claude Tag skills**: project skills committed to a repo also load when that repo is used in a Claude Tag channel

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)